```
!pip install -q gensim
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 27.8/27.8 MB 47.7 MB/s eta 0:00:00
```

```
# Import required libraries

import numpy as np
import tensorflow as tf
import gensim

from tensorflow import keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, Lambda
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

print("TensorFlow version:", tf.__version__)
print("Gensim version:", gensim.__version__)
print("Libraries imported successfully!")
```

```
TensorFlow version: 2.20.0
Gensim version: 4.4.0
Libraries imported successfully!
```

```
# Create an English paragraph of 8 sentences

text = """
Machine learning is an important field of artificial intelligence.
Machine learning helps computers learn from data.
Deep learning uses neural networks to solve complex problems.
Neural networks learn useful patterns from large amounts of data.
Natural language processing helps computers understand human language.
Word embeddings represent words as numerical vectors.
The CBOW model predicts a target word using surrounding context words.
Word2Vec techniques are widely used for learning meaningful word representations.
"""

print(text)
```

```

Machine learning is an important field of artificial intelligence.
Machine learning helps computers learn from data.
Deep learning uses neural networks to solve complex problems.
Neural networks learn useful patterns from large amounts of data.
Natural language processing helps computers understand human language.
Word embeddings represent words as numerical vectors.
The CBOW model predicts a target word using surrounding context words.
Word2Vec techniques are widely used for learning meaningful word representations.
```

```
# Convert text to lowercase
text = text.lower()

# Remove punctuation
import string

text = text.translate(str.maketrans('', '', string.punctuation))

print(text)
```

```

machine learning is an important field of artificial intelligence
machine learning helps computers learn from data
deep learning uses neural networks to solve complex problems
neural networks learn useful patterns from large amounts of data
natural language processing helps computers understand human language
word embeddings represent words as numerical vectors
the cbow model predicts a target word using surrounding context words
word2vec techniques are widely used for learning meaningful word representations
```

```
# Split paragraph into individual sentences

sentences = [
    sentence.strip()
    for sentence in text.split('\n')
    if sentence.strip()
]

print("Total number of sentences:", len(sentences))
```

```
for i, sentence in enumerate(sentences, 1):
    print(i, ":", sentence)
```

```
Total number of sentences: 8
1 : machine learning is an important field of artificial intelligence
2 : machine learning helps computers learn from data
3 : deep learning uses neural networks to solve complex problems
4 : neural networks learn useful patterns from large amounts of data
5 : natural language processing helps computers understand human language
6 : word embeddings represent words as numerical vectors
7 : the cbow model predicts a target word using surrounding context words
8 : word2vec techniques are widely used for learning meaningful word representations
```

```
# Create tokenizer

tokenizer = Tokenizer()

# Fit tokenizer on the sentences
tokenizer.fit_on_texts(sentences)

# Convert sentences into sequences of word indices
sequences = tokenizer.texts_to_sequences(sentences)

print("Tokenized sentences:")
for sequence in sequences:
    print(sequence)
```

```
Tokenized sentences:
[3, 1, 14, 15, 16, 17, 4, 18, 19]
[3, 1, 5, 6, 7, 8, 9]
[20, 1, 21, 10, 11, 22, 23, 24, 25]
[10, 11, 7, 26, 27, 8, 28, 29, 4, 9]
[30, 12, 31, 5, 6, 32, 33, 12]
[2, 34, 35, 13, 36, 37, 38]
[39, 40, 41, 42, 43, 44, 2, 45, 46, 47, 13]
[48, 49, 50, 51, 52, 53, 1, 54, 2, 55]
```

```
# Find vocabulary size and number of sentences

word_index = tokenizer.word_index

total_vocab = len(word_index) + 1
total_sentences = len(sentences)

print("Total number of sentences:", total_sentences)
print("Total number of unique words:", len(word_index))
print("Total vocabulary size:", total_vocab)

print("\nWord Index:")
print(word_index)
```

```
Total number of sentences: 8
Total number of unique words: 55
Total vocabulary size: 56

Word Index:
{'learning': 1, 'word': 2, 'machine': 3, 'of': 4, 'helps': 5, 'computers': 6, 'learn': 7, 'from': 8, 'data': 9, 'neural': 10
```

```
# Set CBOW window size

window_size = 2

print("Window size:", window_size)
print("Maximum context words:", window_size * 2)
```

```
Window size: 2
Maximum context words: 4
```

```
# Generate CBOW context-target pairs

def generate_cbow_data(data, window_size, total_vocab):

    context_data = []
    target_data = []

    for sentence in data:

        sentence_length = len(sentence)

        for idx, word in enumerate(sentence):
```

```
            context_words = []

            begin = idx - window_size
            end = idx + window_size + 1

            for i in range(begin, end):

                if 0 <= i < sentence_length and i != idx:
                    context_words.append(sentence[i])

            # Pad context words to fixed length
            context_words = pad_sequences(
                [context_words],
                maxlen=window_size * 2,
                padding='pre'
            )[0]

            context_data.append(context_words)
            target_data.append(word)

    # Convert target values to one-hot encoding
    target_data = keras.utils.to_categorical(
        target_data,
        num_classes=total_vocab
    )

    return np.array(context_data), np.array(target_data)
```

```
# Generate training data

X, y = generate_cbow_data(
    sequences,
    window_size,
    total_vocab
)

print("Context data shape:", X.shape)
print("Target data shape:", y.shape)
```

```
Context data shape: (71, 4)
Target data shape: (71, 56)
```

```
# Display some context-target examples

reverse_word_index = {
    value: key for key, value in word_index.items()
}

print("Sample CBOW training pairs:\n")

for i in range(min(10, len(X))):

    context_words = [
        reverse_word_index.get(word_id, "<PAD>")
        for word_id in X[i]
        if word_id != 0
    ]

    target_word = reverse_word_index.get(
        np.argmax(y[i]),
        "UNKNOWN"
    )

    print("Context:", context_words)
    print("Target :", target_word)
    print()
```

```
Sample CBOW training pairs:

Context: ['learning', 'is']
Target : machine

Context: ['machine', 'is', 'an']
Target : learning

Context: ['machine', 'learning', 'an', 'important']
Target : is

Context: ['learning', 'is', 'important', 'field']
Target : an

Context: ['is', 'an', 'field', 'of']
Target : important
```

```
Context: ['an', 'important', 'of', 'artificial']
Target : field

Context: ['important', 'field', 'artificial', 'intelligence']
Target : of

Context: ['field', 'of', 'intelligence']
Target : artificial

Context: ['of', 'artificial']
Target : intelligence

Context: ['learning', 'helps']
Target : machine
```

```
# Create CBOW model

embedding_dim = 100

model = Sequential([

    # Convert word IDs into embedding vectors
    Embedding(
        input_dim=total_vocab,
        output_dim=embedding_dim
    ),

    # Average the context word embeddings
    Lambda(
        lambda x: tf.reduce_mean(x, axis=1)
    ),

    # Fully connected layer
    Dense(128, activation='relu'),

    # Output layer
    Dense(total_vocab, activation='softmax')
])

print("CBOW model created successfully!")
```

```
CBOW model created successfully!
```

```
# Build the model

model.build(input_shape=(None, window_size * 2))

model.summary()
```

**Model: "sequential"**

| Layer (type) | Output Shape | Param # |
|---|---|---|
| embedding (Embedding) | (None, 4, 100) | 5,600 |
| lambda (Lambda) | (None, 100) | 0 |
| dense (Dense) | (None, 128) | 12,928 |
| dense_1 (Dense) | (None, 56) | 7,224 |

**Total params:** 25,752 (100.59 KB)
**Trainable params:** 25,752 (100.59 KB)

```
# Compile CBOW model

model.compile(
    loss='categorical_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

print("CBOW model compiled successfully!")
```

```
CBOW model compiled successfully!
```

```
# Train the CBOW model

history = model.fit(
    X,
    y,
```

```
    epochs=100,
    batch_size=16,
    verbose=1
)
```

```
Epoch 72/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.1349
Epoch 73/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.1287
Epoch 74/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.1231
Epoch 75/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.1176
Epoch 76/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.1120
Epoch 77/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.1074
Epoch 78/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.1041
Epoch 79/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0994
Epoch 80/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0953
Epoch 81/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0915
Epoch 82/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0877
Epoch 83/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0846
Epoch 84/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0812
Epoch 85/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0785
Epoch 86/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0755
Epoch 87/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0726
Epoch 88/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0702
Epoch 89/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0678
Epoch 90/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0655
Epoch 91/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0634
Epoch 92/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0612
Epoch 93/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0591
Epoch 94/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 1.0000 - loss: 0.0573
Epoch 95/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0552
Epoch 96/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0536
Epoch 97/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0517
Epoch 98/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0503
Epoch 99/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 1.0000 - loss: 0.0490
Epoch 100/100
5/5 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 1.0000 - loss: 0.0474
```

```
# Plot training loss

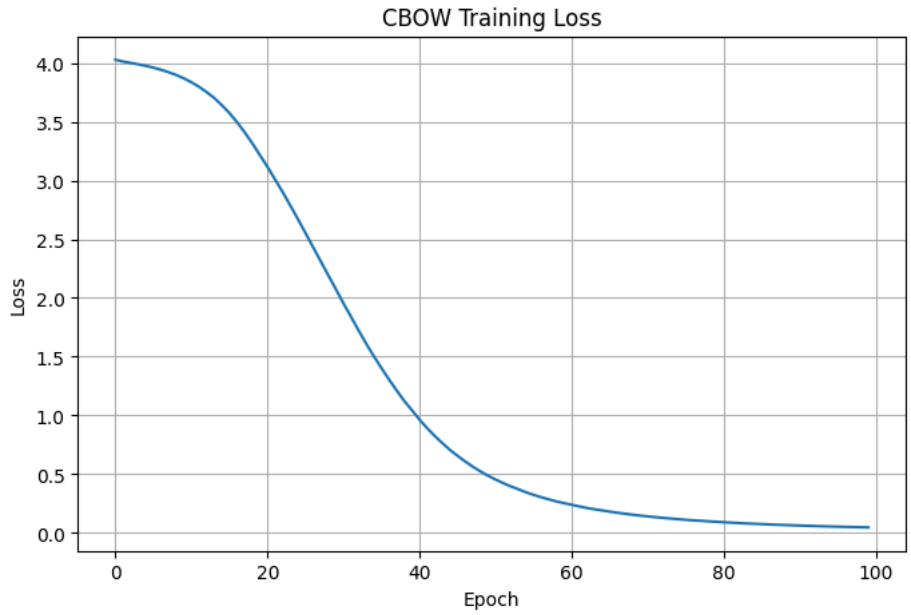
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

plt.plot(history.history['loss'])

plt.title('CBOW Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.grid()

plt.show()
```



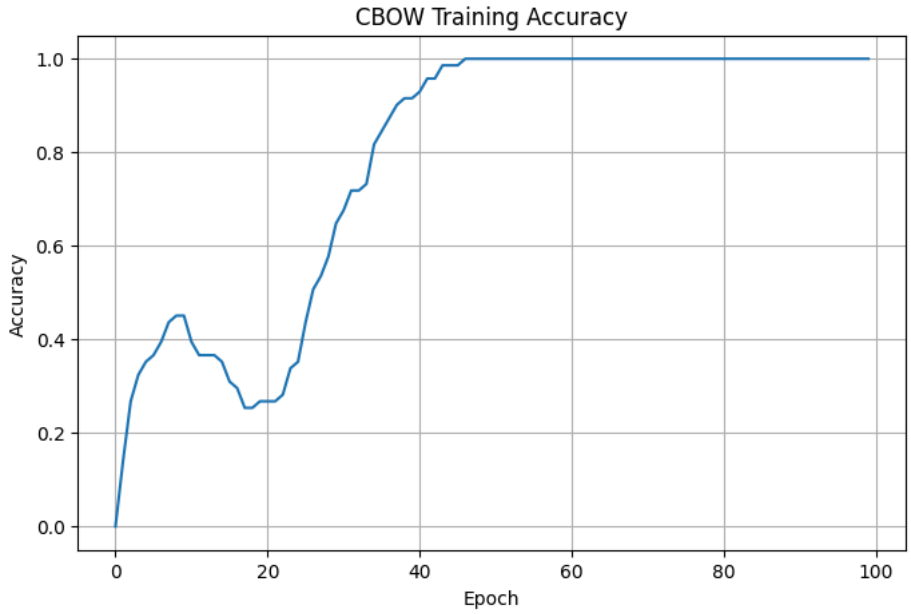

```
# Plot training accuracy

plt.figure(figsize=(8, 5))

plt.plot(history.history['accuracy'])

plt.title('CBOW Training Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.grid()

plt.show()
```




```
# Extract word embedding weights

weights = model.get_weights()[0]

print("Embedding matrix shape:", weights.shape)
print("Embedding dimension:", embedding_dim)
```

```
Embedding matrix shape: (56, 100)
Embedding dimension: 100
```

```
# Create Word2Vec-compatible vector file

vector_file = "/content/vectors.txt"

with open(vector_file, "w") as f:
```

```
        # Header: vocabulary size and vector dimensions
        f.write(f"{total_vocab} {embedding_dim}\n")

        # Write embedding for index 0 (padding/unknown token)
        vector_for_zero = weights[0]
        vector_string_for_zero = " ".join(map(str, vector_for_zero))
        f.write(f"<PAD> {vector_string_for_zero}\n")

        for word, index in word_index.items():

            vector = weights[index]

            vector_string = " ".join(
                map(str, vector)
            )

            f.write(f"{word} {vector_string}\n")

    print("Vector file created successfully!")
    print("File location:", vector_file)
```

```
Vector file created successfully!
File location: /content/vectors.txt
```

```
# Load the trained word vectors

from gensim.models import KeyedVectors

cbow_output = KeyedVectors.load_word2vec_format(
    vector_file,
    binary=False
)

print("Word vectors loaded successfully!")
print("Number of vectors:", len(cbow_output))
print("Vector size:", cbow_output.vector_size)
```

```
Word vectors loaded successfully!
Number of vectors: 56
Vector size: 100
```

```
# Find similar words

word = "learning"

similar_words = cbow_output.most_similar(
    positive=[word],
    topn=5
)

print(f"Words most similar to '{word}':")

for w, score in similar_words:
    print(f"{w}: {score:.4f}")
```

```
Words most similar to 'learning':
representations: 0.4507
widely: 0.4479
is: 0.3510
embeddings: 0.3177
meaningful: 0.2791
```

```
# Test another word

word = "model"

similar_words = cbow_output.most_similar(
    positive=[word],
    topn=5
)

print(f"Words most similar to '{word}':")

for w, score in similar_words:
    print(f"{w}: {score:.4f}")
```

```
Words most similar to 'model':
cbow: 0.6665
predicts: 0.4851
the: 0.3954
target: 0.3717
patterns: 0.2754
```

```
# Display the vector of a word

word = "learning"

vector = cbow_output[word]

print("Word:", word)
print("Vector dimension:", len(vector))
print("\nFirst 10 values of vector:")
print(vector[:10])
```

```
Word: learning
Vector dimension: 100

First 10 values of vector:
[-0.39195442 -0.12050797 -0.05306185 -0.01864449 -0.06923702  0.04187889
  0.4837218  -0.29104093 -0.21721652 -0.16476516]
```

```
# Display final CBOW results

print("=" * 50)
print("CBOW MODEL - FINAL RESULTS")
print("=" * 50)

print("Number of sentences :", total_sentences)
print("Vocabulary size     :", total_vocab)
print("Window size         :", window_size)
print("Embedding dimension :", embedding_dim)
print("Training samples    :", len(X))

print("\nSelected word       :", "learning")

print("\nSimilar words:")
for word, score in cbow_output.most_similar(
    positive=["learning"],
    topn=5
):
    print(f"{word:15s} : {score:.4f}")

print("=" * 50)
```

```
==================================================
CBOW MODEL - FINAL RESULTS
==================================================
Number of sentences : 8
Vocabulary size     : 56
Window size         : 2
Embedding dimension : 100
Training samples    : 71

Selected word       : learning

Similar words:
representations : 0.4507
widely          : 0.4479
is              : 0.3510
embeddings      : 0.3177
meaningful      : 0.2791
==================================================
```

```
Start coding or generate with AI.
```